Course Code: CSA1741 Course Name: ARTIFICIAL INTELLIGENCE

Slot: B

ASSIGNMENT 1 – CO1

Nature of Assignment: Individual Duration: 1 Week Total Marks: 50 Student Name:

Harish Kumar G Reg. No: 192521340

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.
Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

- Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi agent), justifying each choice
- State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

- At least one iterative approach (queue- or stack-based)
- At least one recursive approach
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

- What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)
- Where it occurs in the algorithm

- Why it is appropriate for efficiency analysis
- How the number of times it executes may change as the state-space size increases

Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

Derive the time and space complexity for each proposed approach using mathematical analysis.

Discuss:

- The input size (branching factor *b*, shallowest-goal depth *d*, maximum depth *m*, optimal cost *C**, minimum step cost ε)
- The number of times the basic operation executes, derived by summing the nodes at each level
- The mathematical expression for time and for memory (frontier size) •

Any preprocessing cost involved

- The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4

Marks) Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the best case (for example, the goal lies on the first branch explored)
- What condition produces the average case
- What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

• How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.

Discuss:

- How each algorithm behaves as the state space becomes very large
- How the growth rates of the approaches differ in time and in memory • What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm.

Identify clearly:

- The size of the original problem (depth limit or remaining depth)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.

Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as depth or branching factor increases
- How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them

experimentally. Use generated hospital graphs of approximately:

$1{,}000 \rightarrow 10{,}000 \rightarrow 100{,}000 \rightarrow 1{,}000{,}000$ states

Ensure that:

- All algorithms return correct routes
- The same or equivalent graphs and start/goal pairs are used for comparison • Testing conditions are consistent and the random seeds are reported • Test cases include a reachable goal, an unreachable goal and a start equal to the goal • The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

SIMATS ENGINEERING
SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a

reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes. Record, with code and outputs as proof:

- Nodes generated and nodes expanded
- Maximum frontier size (memory proxy)
- Solution depth and path cost, and whether the path is optimal
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs. **TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)** Compare your theoretical analysis with the experimental results.

Discuss:

- Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity
- Which algorithm scales better as the state space increases
- Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance. **TASK 11 – Recommend and Defend the Final Design (4 Marks)**

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives

SIMATS ENGINEERING
SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations

• Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

• What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?

• Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

• Can your original algorithm still be applied? What does a "state" become? • How large can the new search space be for *k* possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

• Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity • If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

• Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for *k* = 2, 4, 8, with and without a local sensor), with code and

output as proof.

MARK DISTRIBUTION – ASSIGNMENT_1

Task	Marks
Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

SOLUTION – ASSIGNMENT 1 (CO1)

Design, Analysis and Validation of an Efficient Search Strategy for a Hospital Medicine-Delivery Robot

Task 1 – Agent and Search Algorithm Design

1.1 PEAS description

Performance measure	Environment	Actuators	Sensors
Minimum travel time to the ward Medicine delivered to the right ward No collisions, safe around people Low battery use, low planning time	Hospital corridors, junctions Lifts and doors Ward entrances Staff, patients, trolleys (treated as moving obstacles)	Drive wheels and steering Lift-call and door-open interface Locked medicine drawer Display / buzzer for hand-over	Localisation (LiDAR + odometry) Obstacle / bump sensors Camera for ward-door recognition Battery gauge, ward request from the nurse station

1.2 Environment classification

Property	Choice	Justification
Observable	Fully observable (original design)	The hospital map is known and the localisation sensor gives the exact current junction.
Deterministic	Deterministic	Taking corridor (u, v) always ends at v; the travel time is a known number.
Episodic / sequential	Sequential	Each move changes the position, so every decision affects later ones and the total travel time.
Static / dynamic	Static while planning	The route is planned from the stored map before moving; changing crowds are handled by re-planning.
Discrete / continuous	Discrete	Locations form a finite graph of junctions, lifts and doors.
Single / multi agent	Single agent	Only one robot is planned for; people are treated as obstacles, not as competing agents.

1.3 View of AI and history

The robot follows the **acting-rationally** view: it is a goal-based problem-solving agent that picks the action sequence expected to maximise its performance measure (least travel time), given what it knows. It formulates a goal (reach ward W), formulates the problem (graph below), searches, then executes the route. This is the classic state-space search paradigm of early AI: Newell, Simon and Shaw's Logic Theorist (1956) and General Problem Solver (1957–59) treated problem solving as search through a space of states, and shortest-path methods such as Dijkstra's algorithm (1959) are the ancestors of the route-finding done here.

1.4 Problem formulation

Component	Definition
States	V locations of the hospital graph (junction, corridor point, lift stop, ward door); V grows 10^3 → 10^6+.
Initial state	The pharmacy node.
Actions	ACTIONS(u) = {go to v : (u, v) is an edge}; b = branching factor (average out-degree).
Transition model	RESULT(u, go v) = v.
Goal test	State = requested ward node.
Path cost	Sum of edge travel times; step cost c(u, v) ≥ ε > 0 (seconds). Optimal cost = C*.

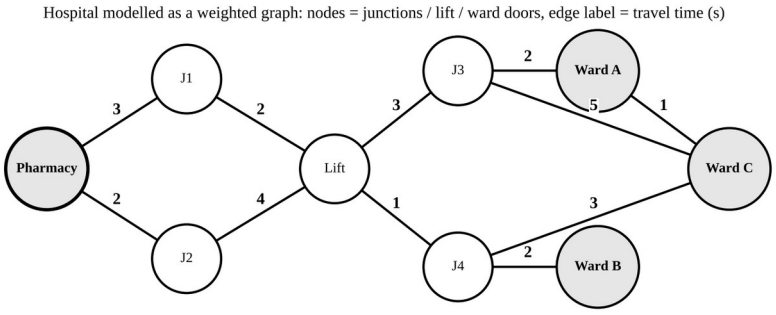


Figure 1: A small hospital graph (edge labels = travel time in seconds).

1.5 Three different uninformed strategies

A1 – Breadth-First Search (iterative, FIFO queue). Idea: expand the shallowest nodes first, so the first time the goal is generated it is reached with the fewest corridors.

```
BFS(adj, s, g):
  if s = g: return [s]
  frontier <- queue[s];  parent <- {s: nil}      // parent also serves as the explored set
  while frontier not empty:
    u <- dequeue(frontier)                        // expansion
    for (v, w) in adj[u]:
      if v not in parent:                         // duplicate check
        parent[v] <- u
        if v = g: return PATH(parent, v)          // goal test at generation
        enqueue(frontier, v)
  return failure
```

A2 – Uniform-Cost Search (iterative, priority queue). Idea: always expand the cheapest frontier node (smallest travel time g(n)); the first time the goal is removed from the queue its cost is the minimum, even if the edge costs differ.

```
UCS(adj, s, g):
  frontier <- min-heap{(0, s)};  dist <- {s: 0};  parent <- {s: nil};  done <- {}
  while frontier not empty:
    (d, u) <- pop-min(frontier)
    if u in done: continue                        // stale heap entry
    if u = g: return PATH(parent, u), d           // goal test at expansion
    add u to done
    for (v, w) in adj[u]:
      if v not in done and d + w < dist.get(v, inf):
        dist[v] <- d + w;  parent[v] <- u;  push(frontier, (d + w, v))
  return failure
```

A3 – Iterative-Deepening DFS (recursive). Idea: run a recursive depth-limited DFS with limit 0, 1, 2, … This combines DFS memory (only the current path) with BFS completeness and fewest-corridor optimality.

```
IDDFS(adj, s, g):
  for limit <- 0, 1, 2, ...:
    seen <- {s: 0};  cutoff <- false
    r <- DLS(s, limit, 0)
    if r is a path: return r
    if not cutoff: return failure                 // nothing cut off: whole component searched
DLS(u, limit, depth):                             // recursive
  if u = g: return [u]
  if depth = limit: cutoff <- true;  return fail
  for (v, w) in adj[u]:
    if seen.get(v, inf) > depth + 1:              // duplicate check by best depth
      seen[v] <- depth + 1
      r <- DLS(v, limit, depth + 1)
      if r != fail: return [u] + r
  return fail
```

Approach	Why it is investigated
A1 BFS	Simplest correct method; fewest corridors; best when edge times are nearly equal and the goal is shallow.
A2 UCS	The edge costs are travel times, so only a cost-ordered strategy can guarantee the fastest route.
A3 IDDFS	Memory stays proportional to depth, which matters when the hospital reaches 10^6+ states; it is the recursive representative.

Preprocessing: (i) build adjacency lists from the edge list – $\Theta(V + E)$ time and space; (ii) an explored/parent hash table (A1, A2) – O(V) space, O(1) average per test; (iii) a per-iteration best-depth table for duplicate detection in A3. These costs are added to every analysis below.

Task 2 – Basic Operation

Algorithm	Basic operation	Where it occurs	Why it is the right one, and how its count grows
BFS	Successor generation with duplicate test (the check "v not in parent" for every edge examined)	Inner for-loop of the expansion step	Every other statement (enqueue, parent update) runs at most once per execution of this test, so time $\propto$ its count. It runs once per outgoing edge of each expanded node: $\approx b \times$ (expansions), growing as b^d until it saturates at 2E.
UCS	Frontier insertion / extraction on the heap (push, pop-min with the relaxation test)	The relaxation line and pop-min	A heap operation costs O(log F) and dominates the constant-time tests around it. Count $\leq$ E pushes and $\leq$ E pops, but it can grow like $b^{C^*/\varepsilon}$ because cheap edges keep the frontier large.
IDDFS	Node expansion, i.e. one call to DLS	Every recursive call	Each call does a goal test, a depth test and a loop over b children – constant work per call. Calls are repeated in every iteration, so the count is Σ over iterations of the nodes within that depth limit.

The basic operation is chosen because it runs most often and the total running time is a constant multiple (or a log factor for UCS) of its execution count. Its count depends on b and d, not directly on V; V only caps it once the search has covered the whole connected part of the hospital.

Task 3 – Time and Space Complexity

Symbols: V = states, E = edges, b = branching factor, d = depth of the shallowest goal, m = maximum depth, C* = optimal cost, ε = minimum step cost. Tree-search counts are derived first; graph-search (with an explored set) bounds follow.

A1 – BFS

Level i holds at most b^i nodes. With the goal test at generation, the worst case generates the whole of level d:

$$\mathbf{T_{BFS} = 1 + b + b^2 + \ldots + b^d = (b^{d+1} - 1)/(b - 1) = \Theta(b^d)}$$

The queue holds a whole level and the explored set holds every generated node, so $S_{BFS} = \Theta(b^d)$. With duplicate detection each state enters once, so $T \le V + E$ (each edge examined twice, once per endpoint) and $S \le V$. Preprocessing: adjacency lists $\Theta(V + E)$. Overall: $\mathbf{\Theta(V + E) + \Theta(\min(b^d, V + E))}$ time and $\Theta(V + E) + \Theta(\min(b^d, V))$ space.

A2 – UCS

Every step costs at least ε, so a node with $g(n) \le C^*$ lies at depth $\le \lfloor C^*/\varepsilon \rfloor$. UCS may therefore expand every node down to depth $1 + \lfloor C^*/\varepsilon \rfloor$:

$$\mathbf{T_{UCS} \le \Sigma_{i=0}^{1+\lfloor C^*/\varepsilon \rfloor} b^i = \Theta(b^{1+\lfloor C^*/\varepsilon \rfloor})\ \text{node operations}}$$

Each costs O(log F) on the heap, giving an extra factor $\log(b^{1+\lfloor C^*/\varepsilon \rfloor}) = (1+\lfloor C^*/\varepsilon \rfloor)\cdot \log b$. Space is the frontier plus explored set, $\Theta(b^{1+\lfloor C^*/\varepsilon \rfloor})$. On a graph, every edge relaxes at most once, so ≤ E pushes and ≤ E pops: $\mathbf{O(E \log E) = O((V + E) \log V)}$ plus $\Theta(V + E)$ preprocessing; space O(V).

A3 – IDDFS

A node at depth i is generated in every iteration with limit ℓ = i, …, d, i.e. (d − i + 1) times:

$$\mathbf{N = \Sigma_{i=0}^{d} (d - i + 1)\cdot b^i = b^d \Sigma_{j=0}^{d} (j + 1)\cdot b^{-j} < b^d / (1 - 1/b)^2 = b^d \cdot (b/(b - 1))^2}$$

(substituting j = d − i and using $\Sigma\,(j+1)x^j = 1/(1 - x)^2$ with x = 1/b). So $N = \Theta(b^d)$. Compared with BFS ($\approx b^d \cdot b/(b - 1)$) the repeated work costs only the constant factor b/(b − 1) – 1.5 for b = 3, 2 for b = 2. Space: one root-to-node path plus pending siblings, $\mathbf{\Theta(b \cdot d)}$ ($\Theta(d)$ if children are generated one at a time). In graph form, the best-depth table lets each node be expanded at most once per distinct depth label (≤ ℓ + 1 times) in iteration ℓ, so total time $\le (V+E)\cdot(1 + 2 + \ldots + (d+1)) = O(d^2(V + E))$ and the table costs O(V) extra space – it removes IDDFS's memory advantage if kept in full.

Task 4 – Best, Average and Worst Case

BFS. *Best:* the pharmacy is the ward (1 node) or the ward is next to it (≤ b + 1 nodes); for deeper goals the best case is the goal being the first node of level d, $\approx b^d/(b - 1)$ generated – only a factor b below the worst case, so d matters far more than position. *Average:* a ward chosen at random lies at the typical depth $\bar{d} \approx \log_b V$, and on average half of that last level is generated: $= \Theta(b^{\bar{d}})$. *Worst:* the goal is the last node of the deepest level, or is unreachable (a ward in a sealed wing): the whole connected component is generated, $\Theta(V + E)$ with duplicates (b^m without).

UCS. *Best:* the cheapest edge from each node leads straight to the goal – about b·log b operations, the same as BFS. *Average:* all nodes whose cost is below C* are expanded, so the work depends on how many cheap corridors surround the pharmacy. *Worst:* **cheap-edge detours** – the goal is one expensive hop away (large C*) while a cloud of ε-cost corridors surrounds the start, so UCS expands everything with $g < C^*$, $b^{1+\lfloor C^*/\varepsilon \rfloor}$ nodes, although d is small; an unreachable ward costs $O((V + E) \log V)$. This is visible in the experiment: at 100,000 states UCS expanded 30,253 nodes against 2,563 for BFS, because the fastest route (39 s) needs 11 corridors while the 8-corridor route takes 41 s.

IDDFS. *Best:* the goal is at depth 0 or 1 (≤ b + 2 nodes); for depth d the best case still repeats all shallower iterations. *Average:* goal depth $\bar{d}$, $N \approx b^{\bar{d}}(b/(b - 1))^2$ weighted by where the goal sits in its level. *Worst:* the goal is the last node of level d, $N = b^d (b/(b - 1))^2$; for an **unreachable** goal the iterations only stop once a pass has nothing to cut off, i.e. after the limit exceeds the largest depth in the component, and nodes can be re-expanded each time they are reached by a shorter path. In my trial runs IDDFS on an unreachable goal at 10,000 states did not finish in minutes, while BFS and UCS finished in milliseconds, so this case was tested only at 1,000 states.

Task 5 – Asymptotic Analysis

Algorithm	Time (tree)	Time (graph)	Space (tree)	Space (graph)
BFS	$\Theta(b^d)$	$\Theta(V + E)$ worst; $O(V + E)$	$\Theta(b^d)$	$\Theta(V)$
UCS	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O((V + E) \log V)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(V)$
IDDFS	$\Theta(b^d)$	$O(d^2(V + E))$	$\Theta(b \cdot d)$	$\Theta(d)$ stack + $O(V)$ table

Both time and space of BFS and UCS are exponential in the **depth** of the goal, not in V. Because a hospital with $V = 10^6$ rooms has only a small depth ($\log_b V \approx$ 13–20 corridors), these algorithms run in seconds, but the frontier is a large fraction

of V (Task 9: 3.1×10^5 and 3.6×10^5 entries at 10^6 states). **Memory therefore becomes the limiting factor first** for BFS and UCS, because the frontier grows with V whereas each entry costs tens of bytes and cannot be avoided. For IDDFS memory stays $\Theta(d)$ (14 stack entries at 10^6 states), so **time** is its limiting factor: it does the same $\Theta(b^d)$ work as BFS with a larger constant (3.2×10^6 node generations against 0.9×10^6). UCS has the extra log factor and the cheap-edge risk, so its time grows fastest when travel times vary widely.

Task 6 – Recurrence for the Recursive Algorithm

Let $T(\ell)$ be the number of basic operations of DLS(u, ℓ), where ℓ = limit − depth is the **remaining depth** (problem size).

- **Subproblem size:** each child call is DLS(v, $\ell - 1$) – size decreases by 1.
- **Number of calls:** one per child, b in the worst case (the branching factor).
- **Work outside the recursion:** goal test, depth test, loop overhead and the duplicate check, a constant c.
- **Base case:** at $\ell = 0$ the call performs the goal test and returns a cutoff: $T(0) = c_0$.

$$\mathbf{T(\ell) = b \cdot T(\ell - 1) + c \quad for\ \ell \geq 1, \quad T(0) = c_0}$$

The recursive stack space obeys $S(\ell) = S(\ell - 1) + O(1)$, $S(0) = O(1)$. For iterative deepening, $I(d) = \Sigma_{\ell=0}^{d} T(\ell)$. On a graph with duplicate detection this recurrence is an upper bound, since pruned children make the actual count smaller.

Task 7 – Solving the Recurrence

The recurrence has the "subtract-one" form $T(n) = aT(n - 1) + c$, not the divide-and-conquer form $T(n) = aT(n/k) + f(n)$, so the Master Theorem does not apply; **back-substitution (iteration)** is the suitable method, and the recursion tree confirms it (Figure 3: level i has b^i calls, each doing work c).

Step 1 – expand: $T(\ell) = b \cdot T(\ell-1) + c = b^2 \cdot T(\ell-2) + c(b + 1) = b^3 \cdot T(\ell-3) + c(b^2 + b + 1) = \ldots$

Step 2 – after k substitutions: $T(\ell) = b^k \cdot T(\ell-k) + c(b^{k-1} + \ldots + b + 1)$. Step 3 – set $k = \ell$:

$$\mathbf{T(\ell) = b^{\ell} \cdot c_0 + c \cdot (b^{\ell} - 1)/(b - 1) = \Theta(b^{\ell})}$$

(Check by induction: if the formula holds for $\ell - 1$, then $b \cdot T(\ell-1) + c = b^{\ell} c_0 + c(b^{\ell} - b)/(b - 1) + c = b^{\ell} c_0 + c(b^{\ell} - 1)/(b - 1)$. ✓) Space: $S(\ell) = \Theta(\ell)$.

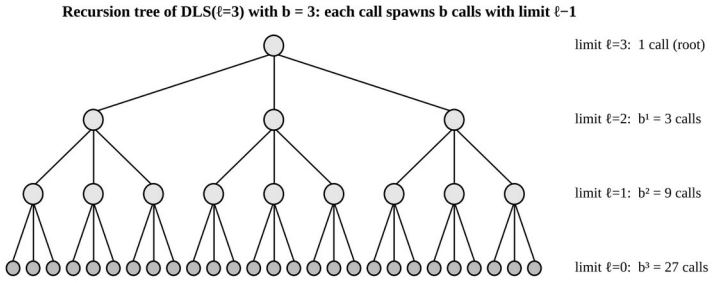


Figure 3: Recursion tree for $T(\ell) = b \cdot T(\ell - 1) + c$ with $b = 3$.

Interpretation. The recursive search is exponential in the depth limit but needs only linear stack space: raising the depth by one multiplies the work by b, while doubling V barely changes it if the depth stays small. **Extension to IDDFS:** with $a = c_0 + c/(b - 1)$, $T(\ell) = a \cdot b^{\ell} - c/(b - 1)$, so

$$\mathbf{I(d) = \Sigma_{\ell=0}^{d} T(\ell) = a \cdot (b^{d+1} - 1)/(b - 1) - c(d + 1)/(b - 1) = \Theta(b^d)}$$

which agrees with Task 3: repeating the shallow iterations costs a constant factor, not an extra exponent. Numerical check: at 10^6 states the goal was at depth 13 and IDDFS generated 3.2×10^6 nodes, i.e. $3.2 \times 10^6 \approx b_eff^{13}$ gives $b_eff \approx 3.2$, close to the mean degree $2E/V = 3.0$ of the generated graph.

Task 8 – Implementation and Testing

All three algorithms were implemented in Python 3 (file search.py, run with python3 search.py). **Test graphs:** a random hospital graph of V = 1,000; 10,000; 100,000; 1,000,000 states, built as a random recursive tree (corridors) plus V/2 extra random links (lifts, bridges), so E = 1.5V, every edge weight is a random travel time from 1 to 9 s. **Seeds:** graph seed 2026; start/goal pair chosen with seed 2026 + V (goal at least 6 corridors away). The same graph and the same pair are given to all three algorithms. Core code:

```
import random, heapq; from collections import deque
def make_graph(n, seed=2026):
    r = random.Random(seed); adj = [[] for _ in range(n)]
    def add(u, v, w): adj[u].append((v, w)); adj[v].append((u, w))
    for i in range(1, n): add(i, r.randrange(i), r.randint(1, 9))
    for _ in range(n // 2):
        u, v = r.randrange(n), r.randrange(n)
        if u != v: add(u, v, r.randint(1, 9))
    return adj
```

```
def bfs(adj, s, g):
    gen, exp, mx = 1, 0, 1; par = {s: None}; dq = deque([s])
    if s == g: return dict(path=[s], gen=1, exp=0, mx=1)
    while dq:
        u = dq.popleft(); exp += 1
        for v, w in adj[u]:
            if v not in par:
                par[v] = u; gen += 1
                if v == g: return dict(path=path_of(par, v), gen=gen, exp=exp, mx=mx)
                dq.append(v)
        mx = max(mx, len(dq))
    return dict(path=None, gen=gen, exp=exp, mx=mx)
def ucs(adj, s, g):
    gen, exp, cnt, mx = 1, 0, 0, 1; dist = {s: 0}; par = {s: None}; pq = [(0, 0, s)]; done = set()
    while pq:
        d, _, u = heapq.heappop(pq)
        if u in done: continue
        done.add(u); exp += 1
        if u == g: return dict(path=path_of(par, u), gen=gen, exp=exp, mx=mx, cost=d)
        for v, w in adj[u]:
            if v not in done and d + w < dist.get(v, 1e18):
                dist[v] = d + w; par[v] = u; cnt += 1; gen += 1; heapq.heappush(pq, (d + w, cnt, v))
        mx = max(mx, len(pq))
    return dict(path=None, gen=gen, exp=exp, mx=mx)
def iddfs(adj, s, g):                    # recursive depth-limited DFS inside an iterative loop
    st = dict(gen=0, exp=0, mx=0, cut=False)
    def dls(u, lim, depth, seen, path):
        st["exp"] += 1
        if u == g: return True
        if depth == lim: st["cut"] = True; return False
        for v, w in adj[u]:
            if seen.get(v, 1 << 30) > depth + 1:
                seen[v] = depth + 1; st["gen"] += 1; path.append(v)
                st["mx"] = max(st["mx"], len(path))
                if dls(v, lim, depth + 1, seen, path): return True
                path.pop()
        return False
    lim = 0
    while True:
        st["cut"] = False; seen = {s: 0}; path = [s]; st["gen"] += 1
        if dls(s, lim, 0, seen, path): return dict(path=path, gen=st["gen"], exp=st["exp"], mx=st["mx"])
        if not st["cut"]: return dict(path=None, gen=st["gen"], exp=st["exp"], mx=st["mx"])
        lim += 1
```

Test results. A route counts as correct when it starts at s, ends at g and every consecutive pair is an edge; the unreachable test uses an isolated extra node as the ward.

States V	Reachable goal (BFS / UCS / IDDFS path length)	Unreachable goal	Start = goal
1,000	PASS: 8 / 8 / 8 corridors	PASS: failure returned	PASS: all return [s], 0 corridors
10,000	PASS: 9 / 9 / 9 corridors	PASS: failure returned (BFS, UCS; IDDFS not run)	PASS: all return [s], 0 corridors
100,000	PASS: 8 / 11 / 8 corridors	PASS: failure returned (BFS, UCS; IDDFS not run)	PASS: all return [s], 0 corridors
1,000,000	PASS: 13 / 16 / 13 corridors	not run	PASS: all return [s], 0 corridors

Note: BFS returns fewest corridors, so at 100,000 and 1,000,000 states its path length (8, 13) is below UCS (11, 16) although UCS is faster in seconds travelled (Task 9).

Task 9 – Experimental Performance

Each row is a mean of 5 timed runs on the same graph and pair (time = search only; graph construction took 1.5 ms, 22 ms, 0.20 s and 2.4 s for the four sizes). For IDDFS "max frontier" is the deepest recursion stack; "Optimal?" means the minimum travel cost found by UCS.

V	Algorithm	Generated	Expanded	Max frontier	Corridors	Travel cost (s)	Optimal?	Time (ms)
1,000	BFS	508	235	272	8	40	Yes	0.19
1,000	UCS	884	517	364	8	40	Yes	1.11
1,000	IDDFS	982	982	9	8	40	Yes	0.45
10,000	BFS	6,311	3,328	2,988	9	34	Yes	2.37
10,000	UCS	6,463	3,107	3,282	9	34	Yes	6.12
10,000	IDDFS	14,934	14,934	10	9	34	Yes	8.11
100,000	BFS	7,414	2,563	4,849	8	41	No	3.12
100,000	UCS	63,775	30,253	32,771	11	39	Yes	135
100,000	IDDFS	11,475	11,475	9	8	41	No	7.91
1,000,000	BFS	914,207	689,926	313,500	13	68	No	1494
1,000,000	UCS	996,390	658,359	357,608	16	51	Yes	4099
1,000,000	IDDFS	3,198,133	3,198,133	14	13	68	No	4503

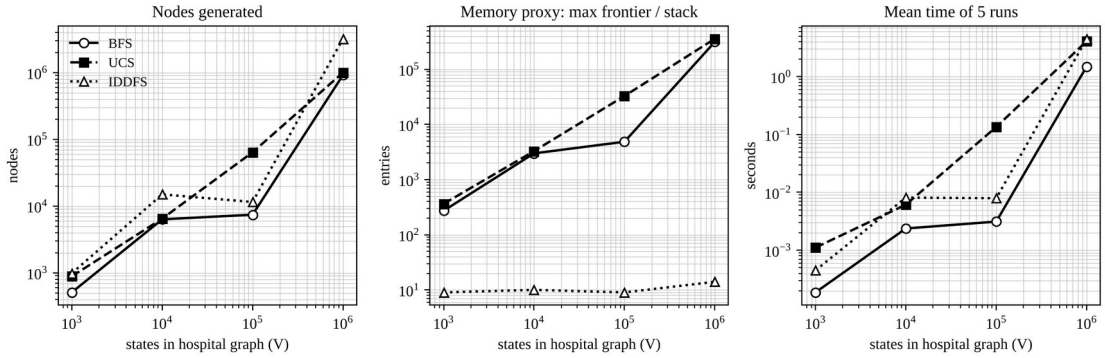


Figure 2: Scaling of the three algorithms with hospital size (log–log axes).

Task 10 – Theory versus Experiment

- **Growth of generated nodes.** Theory says work follows b^d, not V. The data agree: depth stayed between 8 and 13 corridors while V grew 1,000-fold, and BFS generation grew from 508 to 914,207. The 100,000-state point is low (7,414) simply because that goal happened to be only 8 hops away – with one start/goal pair per size the curve is not smooth, which is a limitation of my experiment.
- **Frontier size.** BFS and UCS frontiers grew from 272 and 364 entries ($V = 10^3$) to 313,500 and 357,608 ($V = 10^6$), i.e. roughly $\Theta(V)$ once the search covers a large part of the graph, as predicted for $\Theta(b^d)$ with a saturating graph. The IDDFS stack stayed at 9–14 entries, matching $\Theta(d)$.
- **IDDFS overhead.** Theory predicts $N_{IDDFS}/N_{BFS} \approx b/(b-1)$, between 1.5 and 2. Observed 3.5 at 10^6 states (3,198,133 / 914,207) and 2.4 at 10^4 (14,934 / 6,311). The excess comes from cycles: the graph has $E = 1.5V$, so DLS with a best-depth table sometimes re-expands a node reached again by a shorter path, which the tree analysis ignores; also BFS stops the moment the goal is generated, whereas IDDFS first repeats all the shallower iterations.
- **UCS.** UCS generated 8.6 times more nodes than BFS at 10^5 states and its time grew with an extra log factor (heap), 4.1 s against 1.5 s at 10^6. This is the cheap-edge effect from Task 4, not an error.
- **Which scales better?** In time BFS is best (1.5 s at 10^6); in memory IDDFS is best by four orders of magnitude (14 against 313,500 entries); but only UCS always returns the fastest route. Differences between theory and measurement come from the random graph structure and duplicates, tie-breaking in the queue, Python hash-table and function-call overhead (IDDFS recursion is slower per node than a loop), and the fact that IDDFS's duplicate table (not counted in "max frontier") is itself $\Theta(V \text{ visited})$.

Task 11 – Recommended Design

Recommendation: Uniform-Cost Search with adjacency lists, a binary heap and an explored table.

- **Why not the alternatives.** BFS and IDDFS minimise the number of corridors, not the travel time. In the experiments they returned non-optimal routes at 10^5 states (41 s against 39 s) and 10^6 states (68 s against 51 s, i.e.

33% slower). The robot's performance measure is travel time, so only UCS meets the requirement in all four sizes (optimal = Yes in every row).

- **Efficiency.** Theory $O((V + E) \log V)$; measured 4.1 s for $V = 10^6$ and 0.13 s for $V = 10^5$, which is acceptable for planning once per delivery. Preprocessing is a one-off $\Theta(V + E)$ (2.4 s at 10^6, rebuilt only when wings change).
- **Trade-offs and limits.** UCS has the largest memory ($\approx 3.6 \times 10^5$ frontier entries at 10^6 states) and can slow down with cheap-edge clusters. Practical mitigations that still use uninformed search: because the start is always the pharmacy, run one full UCS from the pharmacy per map update and answer every ward request from the stored parent tree in O(path length) – I did not measure that full run; or use bidirectional UCS. If memory were the only concern, IDDFS would be the fallback for a hop-count route, at the price of a longer travel time.
- **Does it stay practical as the hospital grows?** Up to about 10^6 states yes (seconds). Beyond that, memory grows linearly with V, so the one-off pharmacy-rooted tree or an informed method (A*) would be needed.

Task 12 – Day-4 Design Challenge: Unknown Start Location

A. Impact analysis

The robot now knows only that it is at one of k junctions. Two assumptions break: the environment is no longer **fully observable** (the localisation sensor fails) and the **initial state is no longer known**. The robot's knowledge becomes part of the state because the action that is best from one candidate junction may be wrong from another: the thing being searched is the set of positions the robot might be in (a **belief state**), and the goal is satisfied only when every member of the set is at the dock.

B. Evaluation of the original design

BFS, UCS and IDDFS search over single physical states and cannot be applied directly: a route that works from junction x may not work from y. A state becomes a belief state $b \subseteq S$, so the search space is the power set of S. For k possible start junctions and deterministic moves the set size can only stay the same or shrink, so the reachable belief states number at most $\Sigma_{i=1}^{k} C(V, i) = O(V^k)$, and in the worst case $2^V - 1$. With $V = 10^6$ even $k = 3$ gives about 10^{17} candidate sets, so the original design is **not acceptable as it stands** (the exponent k now sits in the base).

C. Adaptation / redesign

Decision: **REDESIGN the top level, RETAIN the old search as a sub-routine.** (1) Search over belief states: state = frozenset of candidate junctions, initial belief = the k candidates, action a maps b to $\{RESULT(p, a) : p \in b\}$ (bumping into a wall leaves the position unchanged and shrinks the set), goal = {dock}. Breadth-first search with an explored set of frozen belief states gives the shortest sensorless plan. (2) With the local sensor (which of the four neighbouring walls are present), each move is followed by a percept that splits the belief into smaller sets; the plan becomes a conditional plan found by AND–OR search (OR over actions, AND over percepts). (3) As soon as the belief is a single junction, the robot switches back to the original UCS route. Complexity changes from $O((V + E) \log V)$ to $O(V^k)$ belief states without a sensor; the sensor lets each branch handle a smaller belief, so the effort drops sharply.

```
BELIEF-BFS(B0, dock):                                 // sensorless
  frontier <- queue[B0];  seen <- {B0: nil}
  while frontier not empty:
    b <- dequeue(frontier)
    for each action a in {N, S, E, W}:
      b2 <- { RESULT(p, a) : p in b }                 // frozenset, dock is absorbing
      if b2 not in seen:
         seen[b2] <- (b, a)
         if b2 = {dock}: return PLAN(seen, b2)
         enqueue(frontier, b2)
  return failure
SENSOR-PLAN(b, depth):                                // AND-OR, iterative deepening, memoised
  if b = {dock}: return success;   if depth = 0: return fail
  for each action a: b2 <- RESULT(b, a)
     if all SENSOR-PLAN(part, depth-1) for part in SPLIT-BY-PERCEPT(b2): return success
  return fail
```

D. Evidence-based defence

Experiment: 8×8 floor with 50 free junctions and walls (Figure 4), dock in the bottom-right corner, moves N/S/E/W. For each k = 2, 4, 8, five random candidate sets (seed 7) were solved without and with the local sensor. Values are medians of the five trials; "plan length" is the number of moves (for the sensor case the longest branch of the conditional plan). Code: belief.py (BELIEF-BFS and SENSOR-PLAN above).

k	Without sensor: plan length	Without sensor: belief states generated	With sensor: worst-case plan length	With sensor: belief states generated
2	11 (range 9–13)	946	10 (range 7–11)	679
4	14 (range 12–18)	140,499	11 (range 10–12)	1,171
8	no plan found	> 3,000,000 (search aborted)	11 (range 8–13)	1,223

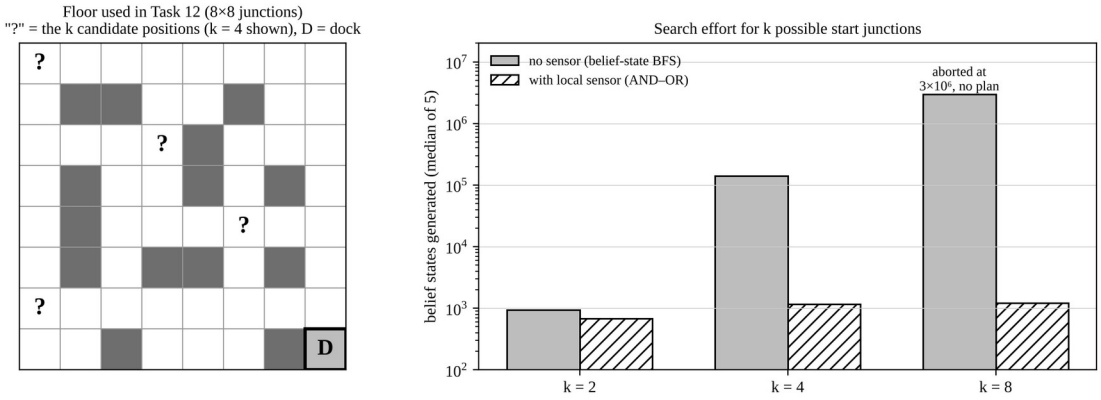


Figure 4: Floor used in the experiment and belief states generated for k = 2, 4, 8.

Reading the evidence. Without a sensor the number of belief states generated grows from about 9×10^2 (k = 2) to 1.4×10^5 (k = 4), about 150 times, and at k = 8 the search exceeded 3×10^6 states without finding a plan: this is the $O(V^k)$ growth predicted in part B, and it confirms that the original design cannot simply be retained. With the local sensor the effort stays near 10^3 states for all k (679, 1,171, 1,223) and the worst-case plan length stays about 10–11 moves, because each percept splits the belief into small sets that are cheap to solve; for k = 2 and 4 the conditional plan is also never longer than the sensorless plan. The redesign (belief-state search with sensing, then the original UCS once the position is known) is therefore justified both mathematically and experimentally. Limitation: this experiment uses a small grid floor with compass moves, not the random hospital graph, so it supports the trend, not exact numbers for the full hospital.